

1. Introduction

Aerospace is a globally interconnected domain that expands rapidly. It scales dynamically as more passengers and aircrafts take on the world of flying. Due to these reasons the current systems in place to manage airline services struggle to keep up, often lagging behind with its reliance on legacy software architectures. The benefit of a flexible system such as cloud computing is a perfect fit to this industry to help with issues of logistics, communication and overall elasticity. It is important to not have down time and system breakdowns in such domains as time sensitivity is crucial. In the case of Aerolink, it is responsible for checking airlines and airport matters across a variety of regions. Aerolink is directly in charge of ensuring many vital services such as passenger ticketing, checking in, baggage tracking as well as the overall flight operations management. All these systems need to run with continuous integration in different areas further highlighting the need to switch to a cloud platform.

1.1 Current Architecture Analysis

The current system used by Aerolink is severely outdated to efficiently carry out all the required functions on time. The legacy architecture is stuck in a single deployable unit that will hinder operations when faults occur. These core issues, as shown in this section, can be addressed by the proper implementation of AWS cloud.

The first problem in the current system is the lack of high availability. This means that if there is a minor error or memory leak the entire system may malfunction. In the case of an airline service any downtime such as this will result in catastrophic delays for flight bookings and checkins disrupting the flight experience. The next issue to be considered is that of scalability. Holiday seasons are extremely busy for flights and will often have more passengers generating a lot of traffic. Scaling a legacy system is incredibly costly and inefficient, whereas AWS cloud will let Aerolink scale up and down depending on the system's current load requirement.

Data synchronization is a must when it comes to the world of airplanes. All ticket data and passenger details must be instantly recorded and updated in the system. Delays must be minimal at worst and the information needs to be updated across international services. AWS cloud can help facilitate this much smoother in comparison to the current system. Furthermore, the tightly coupled architecture means that it is harder to pinpoint where an issue may arise from and the backlog that's created as a result may crash Aerolink's platform, highlighting the need to switch to AWS even more.

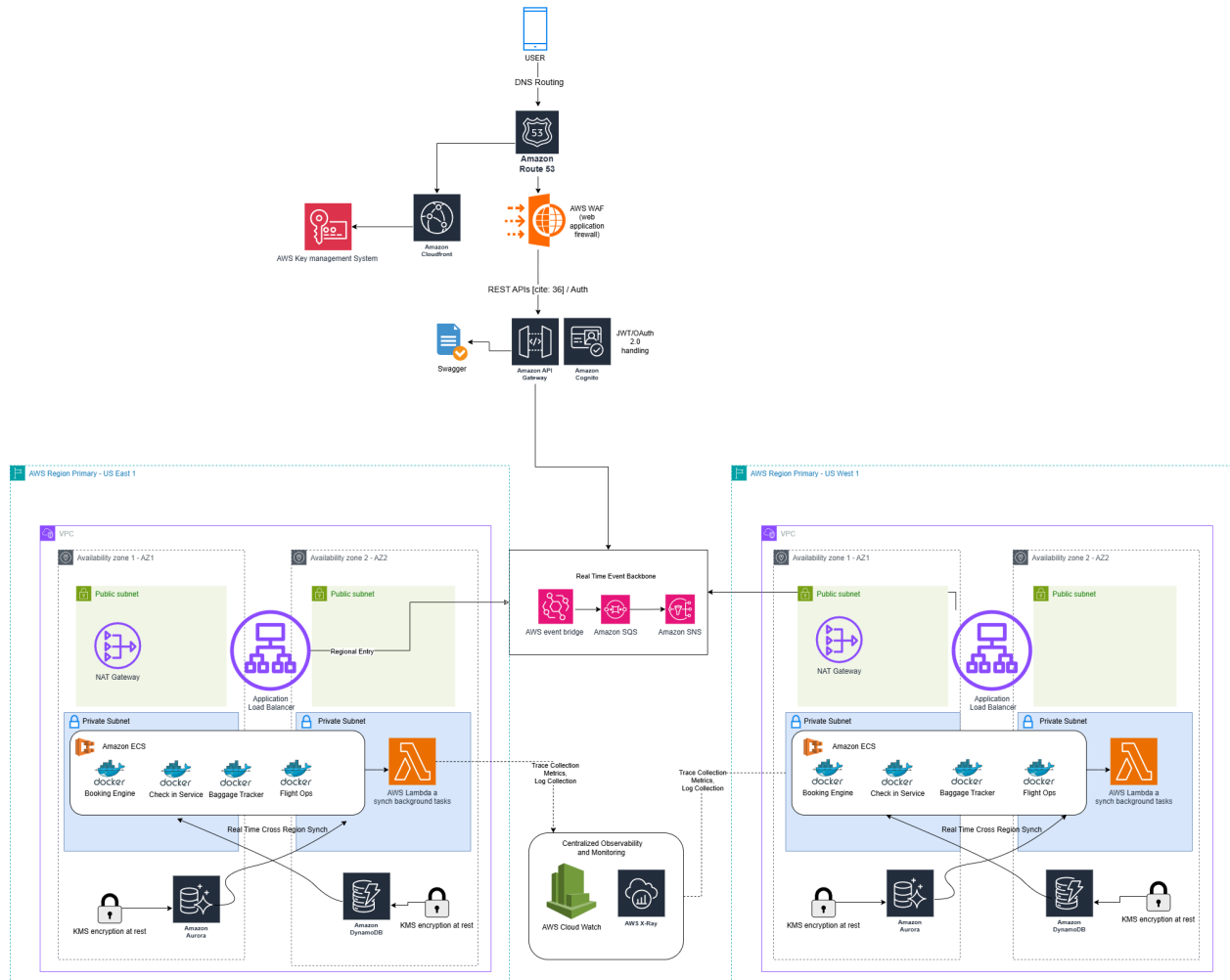
2. Cloud Design Strategy

2.1 General cloud implementation

Aerolink's implemented architecture will be based on the cloud's native microservices strategies that can help to fulfill the daunting demands of running a global network. The idea that is followed in this implementation is one of decoupling the monolithic system into separate microservices that can run independently. These include booking in, checking in, baggage services and overall flight operation.

The switch from dealing with the high operational burden in the old infrastructure to now having to manage cloud services allows Aerolink to focus on business operations smoothly. This system now adheres to the principle of 'high availability' by having a multi-region active deployment model. This aims to avoid the single point of failure problem at both the compute and data layers. Real time data synchronization is also achieved in this model by assisting the event driven backbone of the system for flight scheduling and baggaging. Overall Aerolink becomes scalable, elastic and a stronger platform with these proposed steps.

2.2 Architecture breakdown



2.2.1 Global Network Routing and Edge Security

The system's entry point for any user of the system is strictly managed using a global edge. When a user requests to access the platform, it is firstly resolved by using Amazon Route 53 which uses highly available DNS (Domain Name System) web service to help find the most optimal traffic route for the best geographical location based on overall latency.

No traffic is allowed to enter the system without first passing through the strict security area. Amazon CloudFront is present in the system as a CDN (Content Delivery Network) which assists in securely caching and ensuring proper static delivery of the front end systems and its assets with little latency. In this routing layer, AWS WAF (Web Application Firewall) is also present. WAF is responsible for monitoring the inbound traffic and blocking any suspicious malware such as SQL injections or cross line scripting. AWS WAF overall ensures the system is secure from any external threats.

2.2.2 API Management and Application

The Amazon API Gateway in the architecture acts as the front door for the distributed network after traffic makes its way through the edge security layer. This API layer is responsible for the routing of all requests by RESTful APIs to the required backend micro service. The swagger documentation which contains the API definitions are also present within this layer. Amazon Cognito is also integrated with the API gateway directly and helps to manage user identities as well the handling of generating and validating OAuth 2.0 and JSON Web Tokens (JWT). In the case that an incoming request does not possess a signed JWT the API gateway can directly shut down that request so that unauthenticated traffic does not reach the internal private networks.

2.2.3 Compute Layer and Microservices

Aerolinks business logic to run the platform is present within this layer of the architecture. Virtual Private Clouds (VPCs) operate this section in a highly secure manner. It is split into two high availability zones with **US west 1 and US east 1**, ensuring that there is proper fault tolerance as well.

Traffic that tries to enter into this section is firstly directed at the Application Load Balancer (ALB) which is part of the public subnets and sends the approved traffic into the private subnets that contain the actual micro services. The overall general internet then has no access to these services as a result of this architecture. Amazon ECS (Elastic Container Service) is used here to store the microservices that have been properly containerized using Docker. These microservices

in this section are booking engine , check in service , baggage tracker and flight operations. This system ensures that each microservice can scale up or down as required without interfering with the other services. In order to allow these private containers to download the required updates for the system a NAT gateway is used in the public subnet.

2.2.4 Data Persistence

The implemented new cloud architecture for Aerolink handles the cloud managed database in a few unique ways so that all required data types are accounted for in the system.

- a. ACID or Atomicity, Consistency, Isolation Durability compliance is mandatory for storing transactional data of ticketing, reserving and more in flights. Amazon Aurora and its relational database can be utilized to store these data types. Seats can be managed correctly with no overlapping bookings happening with Aurora intact.
- b. Baggage tracking and real time updates often fall under schemaless and high velocity data and require the use of Amazon DynamoDB. In order to ensure multi region requirements, Aerolink can use DynamoDBs global tables configured to provide active-active synchronisation across all global deployments.
- c. GDPR and PCI-DSS compliances can also be met in this infrastructure by integrating AWS KMS (Key Management System) to make sure that all sensitive data at rest is encrypted.

2.2.5 Event Backbone

A real time event backbone must be in place in the architecture to guarantee that there is no delay present in any real time updates such as updating baggage statuses, for this a Real Time Event backbone is implemented.

If any microservice needs to communicate a state change an event can be sent to the Amazon EventBridge and it can act as the central router and send these events to the Amazon SQS and Amazon SNS systems. SNS (Simple Notification Service) can immediately send out

broadcasting while the SQS (Simple Queue System) can queue the message as required. These systems are primarily in place to build up more fault tolerance so that even in the event a service is offline it can store this message offline temporarily. Moreover, AWS Lambda functions can be used here for serverless computing in order to process necessary light weight background tasks.

2.2.6 Centralized Observing

Telemetry data of the system must go to a centralized observability system to constantly monitor the health and performance of the system. Two different Amazon services are used for this purpose in Aerolink. Amazon CloudWatch is firstly used to get logs on the performance of the entire architecture, followed by AWS X-Ray that allows for distributed tracing to track a single user request as it goes around the API Gateway, the ECS containers and the databases to quickly identify any potential bottlenecks and faults.

References